TP1 : cadrer une application IA et préparer l'environnement

Énoncé détaillé

A. Larhlimi

Développement et déploiement d'applications intelligentes

Semaine 1 · TP de 2 h · en binôme

Plan

Introduction

Préparation

Architecture du dépôt

Partie A : cadrage

Partie B : environnement et dépôt

Les scripts et les tests

Cas problèmes

Ce que vous rendez

Plan

Introduction

Préparation

Architecture du dépôt

Partie A : cadrage

Partie B : environnement et dépôt

Les scripts et les tests

Cas problèmes

Ce que vous rendez

Les deux objectifs de la séance

Cadrer avant de coder

La première question n'est pas « quel modèle ? » mais « faut-il vraiment un modèle ? ».

Trois cas à trancher avec une grille à quatre critères, et à justifier par écrit.

Un dépôt qui survit à votre portable

Un modèle dans un notebook n'est pas un produit : personne ne peut l'exécuter ni le tester.

À la fin : un dépôt que l'enseignant clone et fait tourner en trois commandes.

Ce TP ne produit aucune fonctionnalité visible. Il produit les conditions de tout le reste.

Ce que vous saurez faire, et comment c'est vérifié

Compétence	Vérifiée par
Choisir entre règles métier, ML, RAG et génératif, et le justifier	12 tests de cadrage, lecture des fiches
Créer un environnement Python isolé et reproductible	23 tests d'environnement
Travailler à deux sur un dépôt Git sans vous écraser	3 tests d'historique
Faire tourner une intégration continue	La pastille verte de l'onglet Actions

Soit 38 tests, qui tourneront encore à chaque TP jusqu'en semaine 11.

L'application fil rouge

Une équipe relation client reçoit des milliers d'avis par jour et les traite dans l'ordre d'arrivée. On veut faire remonter les **avis négatifs** en tête de file.

Sem.	Ce qui s'ajoute	Sem.	Ce qui s'ajoute
1	cadrage, environnement, dépôt, tests	6 à 8	paquet, Docker, CI/CD
2	le modèle (TF-IDF, régression logistique)	9	suivi de la dérive, réentraînement
3	une API REST	10	agents : le modèle devient un outil
5	une interface de démonstration	11	projet et soutenance

Problème simple côté modèle, pour que la difficulté soit là où elle est vraiment : le déploiement, les tests, le suivi.

Déroulé et contrôles

	Étape	Qui	Contrôle
	Décompresser le kit (5 min)	chacun	T0 : 7 dossiers, `.gitignore` présent
B1	Environnement virtuel (10 min)	chacun	T1 : 3 failed, 31 passed, 4 skipped
A	Cadrage, trois fiches (50 min)	le binôme	T2 : 12 passed
B2-B3	Dépôt GitHub et push (25 min)	membre A	T3 : aucune ligne `.venv/`
B4	Installation complète (15 min)	chacun	T4 : 0 erreur(s)
B5	Un commit par membre (10 min)	chacun	T5 : deux auteurs, 3 passed
B6	Vérification finale (10 min)	le binôme	T6 : 38 passed, CI verte
B7	Clone frais (5 min)	le binôme	T7 : 38 passed sans intervention

Ne passez à l'étape suivante qu'une fois le contrôle précédent conforme.

Plan

Introduction

Préparation

Architecture du dépôt

Partie A : cadrage

Partie B : environnement et dépôt

Les scripts et les tests

Cas problèmes

Ce que vous rendez

Prérequis et réglages, avant la séance

Objectif

Arriver avec un poste prêt. Ces quatre lignes suppriment à elles seules trois des blocages les plus fréquents du TP.

```
python --version                          # 3.11.x attendu ; sinon py -3.11 --
    version
git --version                             # si inconnu : rouvrir le terminal

git config --global user.name "Prénom Nom"          # nom compté par le contrô
    le T5
git config --global user.email "prenom.nom@ecole.ma"
git config --global init.defaultBranch main         # évite de renommer master
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned    # autorise le venv
```

Python 3.11 de préférence : 3.12 et 3.13 n'ont pas de paquet précompilé pour les versions figées.

Rôles du binôme et récupération du kit

Rôle	Ce qu'il fait seul	Ce qu'il fait comme l'autre
Membre A	Crée le dépôt GitHub, pousse le kit (B2, B3)	venv, installation, son commit, vérifications
Membre B	Clone le dépôt une fois créé (B4)	venv, installation, son commit, vérifications

```
unzip sentiment-app.zip          # Windows : clic droit, Extraire tout
cd sentiment-app                 # toutes les commandes du TP partent d'ici
ls ; ls -Force | findstr gitignore
```

Contrôle T0 : le kit est complet

Sept dossiers, six documents, `requirements.txt`, `pytest.ini`, et `.gitignore`, qui ne s'affiche pas avec un `ls` ordinaire. C'est le seul rempart contre l'erreur de T3 : on le vérifie maintenant.

Plan

Introduction

Préparation

Architecture du dépôt

Partie A : cadrage

Partie B : environnement et dépôt

Les scripts et les tests

Cas problèmes

Ce que vous rendez

Les sept dossiers

Dossier	Contient	Se remplit
`data/`	Jeux de données	Semaine 2, versionné par DVC en semaine 7
`models/`	Modèles entraînés	Semaine 2. Jamais dans Git : lourd, reconstructible
`app/`	API FastAPI	Semaine 3
`ui/`	Interface Streamlit	Semaine 5
`tests/`	Tests pytest	Dès aujourd'hui : 38 tests
`scripts/`	Installation et diagnostic	Fourni
`docs/cadrage/`	Fiches de cadrage	Partie A de ce TP

Les dossiers vides contiennent un fichier `.gitkeep`, vide lui aussi : **Git ne versionne pas les dossiers, seulement les fichiers**. Sans ce témoin, la structure ne serait pas la même chez tout le monde après un clone.

Deux fichiers qui gouvernent tout le reste

requirements.txt

```
pandas==2.2.*
scikit-learn==1.5.*
joblib==1.4.*
pyarrow==21.0.*
fastapi==0.115.*
streamlit==1.39.*
```

==2.2.* accepte 2.2.0 ou 2.2.3, jamais 2.3 : les correctifs passent, les changements de comportement non.

Tout le semestre est installé dès aujourd'hui, pour ne plus jamais y revenir.

.gitignore

```
.venv/              # 100s de Mo
__pycache__/        # bytecode
.env                # secrets
models/*            # artefacts
!models/.gitkeep    # exception
data/*
!data/sample_reviews.csv
```

Le ! inverse une règle. La mécanique sépare ce qui est **lourd et reconstructible** (exclu) de ce qui est **léger et partagé** (inclus).

Les autres fichiers du kit

Fichier	Rôle
`.env.example`	Modèle des variables secrètes. Versionné car vide de vrais secrets ; chacun le copie en `.env`, lui-même ignoré
`pytest.ini`	`testpaths = tests` évite de parcourir tout le dépôt ; `addopts = -ra` résume échecs et tests ignorés
`.editorconfig`	UTF-8, fins de ligne Unix, indentation : évite les conflits illisibles entre Windows et macOS
`.vscode/settings.json`	Sélectionne l'interpréteur `.venv` et active pytest : supprime la cause n°1 du `ModuleNotFoundError`
`Makefile`	Raccourcis `make setup`, `make check`, `make test` (Linux, macOS, Git Bash)
`README.md`	Porte d'entrée : installation, structure, tableau du binôme
`MARCHE-A-SUIVRE.md`	Le pas à pas, huit étapes et huit contrôles
`TESTS.md`, `AIDE-GIT.md`, `DEPANNAGE.md`	Tests en détail, commandes Git, messages d'erreur

Plan

Introduction

Préparation

Architecture du dépôt

Partie A : cadrage

Partie B : environnement et dépôt

Les scripts et les tests

Cas problèmes

Ce que vous rendez

La grille à quatre critères

Objectif

Décider, pour trois besoins métier, quelle approche employer, et savoir le justifier. Les frameworks changent ; le raisonnement de cadrage reste.

	Critère	**La question à se poser**
(a)	Données étiquetées	A-t-on des exemples annotés, ou peut-on en produire à faible coût ?
(b)	Vérifiabilité	La réponse doit-elle être tracée jusqu'à une source ou une règle ?
(c)	Coût par requête	Combien d'appels par jour, et que coûte chacun ?
(d)	Conséquence d'une erreur	Que se passe-t-il concrètement quand le système se trompe ?

L'arbre de choix

Parcourez les questions dans l'ordre ; la première réponse « oui » donne l'approche.

1. Logique connue, stable, devant rester explicable ? → **règles métier**
2. Données étiquetées pour une tâche répétitive ? → **machine learning**
3. Réponse à tirer de documents existants, et à citer ? → **RAG**
4. Sinon, et seulement sinon → **modèle génératif seul**

Approche	**Coût par requête**	**Vérifiabilité**
Règles métier	quasi nul	totale, la règle est lisible
Machine learning	très faible	moyenne, on explique par les exemples
RAG	moyen	bonne, la source est citée
Génératif seul	élevé	faible, d'où la relecture humaine

Les trois cas à cadrer

Fiche	Le cas	Questions à trancher
`cas1-pharmacie.md`	Détecter les ordonnances incomplètes	Les champs obligatoires sont-ils connus ? Que coûte une ordonnance validée à tort ?
`cas2-reglement.md`	Répondre sur le règlement intérieur	Les documents existent. La réponse doit-elle être vérifiable ?
`cas3-avis-negatifs.md`	Prioriser les avis clients négatifs	Des milliers d'avis. Peut-on les étiqueter ? Que coûte un avis mal classé ?

Douze minutes par cas. **Un membre tient le clavier, l'autre parcourt l'arbre de choix à voix haute et conteste.** Inversez à chaque fiche : en binôme, personne ne reste spectateur.

Anatomie d'une fiche

```
## Besoin en une phrase (obligatoire)

> Remplir chaque rubrique en une à trois phrases.    <- consigne, à laisser

Signaler au pharmacien toute ordonnance à laquelle il manque un champ
    obligatoire.

## Approche retenue (obligatoire)

- [x] Règles métier          <- une seule croix dans tout le fichier
- [ ] Machine learning
- [ ] RAG
- [ ] Modèle génératif seul

## Justification (obligatoire)

(a) Aucune ordonnance étiquetée n'existe au départ. (b) Le pharmacien doit
    pouvoir
expliquer le refus. (d) Une ordonnance validée à tort a une conséquence
    grave.
```

Huit rubriques obligatoires, une facultative : l'approche écartée et pourquoi.

Les trois règles, et le contrôle

1. **Une seule case cochée** par fiche : remplacez l'espace par un `x`, sans toucher au reste de la ligne.
2. **Au moins deux critères cités par leur lettre**, écrits tels quels : `(a)`, `(b)`, `(c)`, `(d)`.
3. **Chaque rubrique obligatoire** contient une phrase complète, soit 20 caractères hors consigne.

```
python -m pytest tests\test_cadrage.py -q        # attendu : 12 passed
```

L'erreur qui coûte le plus de temps

Écrire « il y a des données étiquetées » **sans** la lettre `(a)` fait échouer le test. Les lettres doivent apparaître littéralement dans la justification.

Les réponses attendues

Cas	Approche	Pourquoi
Pharmacie	Règles métier	(a) pas d'étiquettes, (b) explicabilité exigée, (d) erreur grave
Règlement	RAG	(b) la réponse cite l'article, (a) pas de questions étiquetées, (c) volume faible
Avis négatifs	Machine learning	(a) étiquettes gratuites via la note, (c) milliers d'avis, (d) erreur peu grave

Ce qui est noté

Deux approches peuvent se défendre pour un même cas : **c'est la justification qui est notée, pas la case cochée.** Le machine learning reste acceptable pour la pharmacie si le binôme précise qu'il faut d'abord constituer un jeu étiqueté et conserver la validation du pharmacien.

Plan

Introduction

Préparation

Architecture du dépôt

Partie A : cadrage

Partie B : environnement et dépôt

Les scripts et les tests

Cas problèmes

Ce que vous rendez

B1. L'environnement virtuel

Objectif

Installer les paquets du cours sans toucher au Python du système, et pouvoir reconstruire cet environnement en une commande sur n'importe quelle machine.

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1          # Linux, macOS : source .venv/bin/
    activate
python -m pip install pandas scikit-learn pytest
```

`-m venv` exécute le module livré avec Python ; `py -3.11` choisit la version.
L'activation modifie le PATH du terminal courant : l'invite prend `(.venv)`. À refaire dans chaque nouveau terminal, y compris celui de VS Code.
`python -m pip` plutôt que `pip` : passe par l'interpréteur courant, ne dépend pas du PATH.

Trois paquets seulement : l'installation complète vient en B4.

B1. Contrôle T1, qui doit être rouge

```
python -m pytest -q        # attendu : 3 failed, 31 passed, 4 skipped
```

Ce que vous voyez	Pourquoi	Résolu à l'étape
3 failed, dans `test_cadrage.py`	Les fiches sont vides	Partie A
4 skipped	Ces tests attendent un dépôt Git	B3 et B5
31 passed	Structure, paquets, `.gitignore` en place	déjà bon

Si vous obtenez 38 passed ici, vous testez un autre dossier. Un test rouge avant que le travail soit fait est un test utile ; un test vert dès le départ ne vérifie rien.

B2. Créer le dépôt sur GitHub

Objectif

Donner au binôme un point de rendez-vous unique, d'où l'enseignant pourra cloner.

1. **New repository** : nom `sentiment-app`, visibilité **Private**.
2. **Ne cochez rien** dans « Initialize this repository with » : ni README, ni `.gitignore`, ni licence. Le kit les fournit, et un dépôt pré-rempli provoquerait un conflit au premier push.
3. **Settings, Collaborators, Add people** : membre B en **Write**, enseignant en **Read**.
4. Copier l'URL HTTPS depuis le bouton vert **Code**.

L'invitation doit être acceptée

Le membre B reçoit un courriel. Tant qu'il ne clique pas, son clone et son push échouent avec `403 Forbidden`.

B3. Initialiser, et le contrôle T3

```
git init            # crée le dépôt local, c'est-à-dire le dossier caché .git/
git add -A          # met tout en attente, sauf ce qu'exclut .gitignore
git status          # CONTRÔLE T3 : lire la sortie AVANT de valider
```

T3 : le seul contrôle irréversible du TP

La sortie ne doit contenir **aucune ligne commençant par** `.venv/`. Sinon, arrêtez-vous : le `.gitignore` manque. Une fois poussé, le dépôt pèse 300 Mo, GitHub refuse les fichiers de plus de 100 Mo, et `test_venv_never_committed` reste rouge même après un `git rm -cached`, car il lit **tout l'historique**. La seule correction est de refaire le dépôt.

Profitez-en pour repérer tout fichier parasite : archive, PDF, capture d'écran.

B3. Pousser, et s'authentifier

```
git commit -m "Semaine 1 : structure du depot et kit TP1"
git branch -M main                    # GitHub attend main
git remote add origin https://github.com/votre-compte/sentiment-app.git
git remote -v                         # vérifiez : PAS de chevrons dans l'URL
git push -u origin main
```

Le mot de passe est refusé

Il faut un **jeton d'accès personnel** : Settings, Developer settings, Personal access tokens, Tokens (classic), portée `repo`. Copiez-le tout de suite, il ne s'affiche qu'une fois.

Deux raccourcis

`bash scripts/init_depot.sh <url>` enchaîne ces commandes et refuse de pousser si `.venv/` est en attente.

`gh auth login` évite les jetons si GitHub CLI est installé.

B4. Installer l'environnement complet

Objectif

Que les deux postes aient exactement le même environnement, par une seule commande.

```
git clone https://github.com/votre-compte/sentiment-app.git    # membre B
cd sentiment-app
powershell -ExecutionPolicy Bypass -File scripts\setup.ps1     # Windows
bash scripts/setup.sh                                          # Linux, macOS, Git Bash
```

Le script enchaîne quatre étapes : vérifier Python, créer `.venv`, installer `requirements.txt` puis figer l'état dans `requirements.lock`, lancer le diagnostic. Comptez 5 à 10 minutes, c'est tout le semestre qui s'installe.

Fichier	Qui l'écrit	Contenu	Dans Git ?
`requirements.txt`	L'enseignante	`pandas==2.2.*`	Oui, c'est le contrat
`requirements.lock`	`pip freeze`, chez vous	`pandas==2.2.3`	Oui, par UN SEUL membre

B4. Contrôle T4, le diagnostic

```
python scripts\check_setup.py
```

Niveau	Signification
`OK`	Rien à faire
`ATTENTION`	Ne bloque pas, mais doit disparaître avant la fin de la séance
`ERREUR`	Bloque : venv non activé, paquet absent, dossier manquant, pas de dépôt Git, `.venv/` suivi

Contrôle T4

0 erreur(s). Un ou deux avertissements sont normaux ici : les fiches si la partie A n'est pas finie, et le fait qu'un seul membre ait commité.

B5. Un commit par membre

Objectif

Que l'historique prouve que les deux membres ont travaillé, et que chacun ait vécu le cycle pull, commit, push.

```
git pull                              # récupère les commits de l'autre AVANT de
    pousser
git add README.md                     # un seul fichier nommé, pas -A
git commit -m "README : ajout de <nom> au tableau du binome"
git push
git shortlog -sn --all                                  # T5 : deux noms distincts
python -m pytest tests\test_git_history.py -q           # T5 : 3 passed
```

Pourquoi nommer le fichier : `git add -A` embarquerait le `requirements.lock` régénéré chez vous, qui provoquerait un conflit chez l'autre.
En cas de conflit sur `README.md` : gardez les deux lignes, supprimez les marqueurs «««<, =======, »»»>, puis `add`, `commit`, `push`.

Si le test dit « 1 auteur » : soit un membre n'a pas poussé, soit les deux postes partagent le même `user.name`.

B6. Vérification finale et intégration continue

```
git pull
python scripts\check_setup.py       # T6 : 0 erreur(s), 0 avertissement(s)
python -m pytest -q                 # T6 : 38 passed
```

Paramètre de `ci.yml`	**Rôle**
`on: [push, pull_request]`	Les tests tournent à chaque push
`fetch-depth: 0`	Récupère TOUT l'historique : sinon `git shortlog` ne verrait qu'un auteur
`python-version: "3.11"`	La version de référence, quelle que soit la vôtre
venv créé dans chaque `run`	Sinon `test_virtualenv_active` échouerait

Une CI verte veut dire que le dépôt fonctionne **ailleurs que sur vos deux portables**.

B7. Le test de l'enseignant

Objectif

Vérifier que le dépôt tient debout tout seul, puisque c'est ainsi qu'il sera noté.

```
cd $env:TEMP                                                    # Linux, macOS : cd /tmp
git clone https://github.com/votre-compte/sentiment-app.git verif
cd verif
py -3.11 -m venv .venv ; .\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python scripts\check_setup.py
python -m pytest -q
```

Contrôle T7

0 erreur(s) et **38 passed**, sans aucune intervention manuelle : aucun fichier à créer, aucune commande à ajouter. Si quelque chose manque ici alors que tout marchait chez vous, c'est qu'un fichier n'a jamais été poussé.

Raccourci : `bash scripts/verif_clone.sh <url>`, qui nettoie tout seul.

Plan

Introduction

Préparation

Architecture du dépôt

Partie A : cadrage

Partie B : environnement et dépôt

Les scripts et les tests

Cas problèmes

Ce que vous rendez

Ce que vérifie `check_setup.py`

Objectif

Comprendre ce que la machine contrôle, pour corriger seul au lieu d'attendre.

```
ROOT = Path(__file__).resolve().parents[1]
in_venv = sys.prefix != getattr(sys, "base_prefix", sys.prefix)
MIN_AUTHORS = 2
```

Fonction	Ce qu'elle contrôle
`check_python`	Version, et venv actif par comparaison de `sys.prefix` et `sys.base_prefix`
`check_packages`	`pandas`, `sklearn`, `pytest` s'importent. On installe `scikit-learn`, on importe `sklearn`
`check_structure`	Sept dossiers, trois fichiers requis, cinq motifs du `.gitignore`
`check_git`	Git présent, `.git/` existe, auteurs via `git shortlog`, `.venv/` absent des fichiers suivis
`check_cadrage`	Trois fiches `cas*.md`, exactement un `- [x]` dans chacune

Les 38 tests

Fichier	Tests	Ce qu'il vérifie
`test_environment.py`	23	Version Python, venv actif, trois paquets, sept dossiers, quatre fichiers, versions figées, cinq motifs du `.gitignore`, rien de secret commité
`test_cadrage.py`	12	Trois fiches, huit rubriques, rubriques remplies, une seule case cochée, deux critères cités
`test_git_history.py`	3	Au moins un commit, deux auteurs distincts, `.venv/` absent de tout l'historique

`conftest.py` fournit deux fixtures partagées, `root` et `cadrage_dir` : un test qui déclare un paramètre `root` en reçoit la valeur. C'est l'injection de dépendances de pytest.

Les tests Git sont ignorés, pas en échec, tant qu'il n'y a pas de dépôt : c'est pourquoi T1 affiche `4 skipped`.

Les paramètres réglables

Paramètre	Fichier	Valeur	Effet si on le change
`MIN_AUTHORS`	`check_setup.py`, `test_git_history.py`	2	Nombre d'auteurs exigés
`REQUIRED_DIRS`	`check_setup.py`, `test_environment.py`	7 dossiers	La structure attendue
`GITIGNORE_MUST_CONTAIN`	`check_setup.py`	5 motifs	Les motifs vérifiés
`REQUIRED_SECTIONS`	`test_cadrage.py`	8 rubriques	Les titres attendus dans les fiches
`CRITERIA`	`test_cadrage.py`	(a) à (d)	Les lettres acceptées

À ne pas faire

Ces constantes sont en haut de chaque fichier, lisibles sans chercher. Ne les modifiez pas pour faire passer un test : c'est exactement ce que la notation regarde.

Plan

Introduction

Préparation

Architecture du dépôt

Partie A : cadrage

Partie B : environnement et dépôt

Les scripts et les tests

Cas problèmes

Ce que vous rendez

Les messages que vous rencontrerez

Message	Correction
`pytest n'est pas reconnu`	Activer le venv, ou écrire `python -m pytest`
`Activate.ps1 cannot be loaded`	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
`ModuleNotFoundError: pandas`	Venv non activé : `where python` doit contenir `.venv`
`No matching distribution found`	Python 3.12 ou 3.13 : installer 3.11, recréer le venv
`error: 400` sur l'URL	Les chevrons de l'exemple sont restés dans l'adresse
`Repository not found`	Dépôt non créé sur GitHub, ou faute de frappe
`Password authentication is not supported`	Jeton d'accès personnel, ou `gh auth login`
`403 Forbidden`	Invitation de collaborateur non acceptée
`rejected, non-fast-forward`	`git pull`, résoudre le conflit, `git push`
`1 auteur(s) de commits, 2 attendus`	`user.name` distinct sur chaque poste, puis un vrai commit
`rubriques vides ou trop courtes`	Remplir la rubrique nommée dans le message
`4 skipped` alors que le dépôt existe	pytest lancé hors du dossier du dépôt

Deux situations à connaître avant de les vivre

Le dépôt qui pèse 300 Mo

`.venv/` a été commité. Le test lit **tout l'historique**, donc un `git rm -cached` tardif ne suffit pas.

Correction au TP1 : supprimer `.git`, vérifier le `.gitignore`, refaire `init`, `add`, `status`, `commit`, puis `push -force`. L'autre membre reclone.

Leçon : `git status` avant chaque premier commit.

Authentification ou autorisation ?

`Password authentication is not supported` : GitHub ne sait pas **qui** vous êtes. Réponse : un jeton.

`403 Forbidden` : GitHub le sait, mais vous n'avez pas le **droit** d'écrire. Réponse : l'invitation de collaborateur.

Leçon : lire le mot qui suit le code d'erreur avant de réessayer.

En cas de blocage : la commande exacte, le message complet, et la sortie de `check_setup.py`. Ces trois éléments suffisent presque toujours.

Plan

Introduction

Préparation

Architecture du dépôt

Partie A : cadrage

Partie B : environnement et dépôt

Les scripts et les tests

Cas problèmes

Ce que vous rendez

La structure finale sur GitHub

```
sentiment-app/                      dépôt GitHub PRIVÉ du binôme
|-- README.md                       tableau du binôme COMPLÉTÉ
|-- TP1-ENONCE.md MARCHE-A-SUIVRE.md TESTS.md AIDE-GIT.md DEPANNAGE.md
|-- requirements.txt                NE PAS MODIFIER
|-- requirements.lock               GÉNÉRÉ, commité par UN SEUL membre
|-- .gitignore .env.example .editorconfig pytest.ini
|-- Makefile
|-- .github/workflows/ci.yml        intégration continue
|-- .vscode/                        interpréteur et pytest préconfigurés
|-- docs/cadrage/
|   |-- cas1-pharmacie.md           À REMPLIR  (livrable noté)
|   |-- cas2-reglement.md           À REMPLIR  (livrable noté)
|   |-- cas3-avis-negatifs.md       À REMPLIR  (livrable noté)
|   |-- TEMPLATE, EXEMPLE-corrige, GRILLE, BANQUE-DE-CAS
|-- scripts/    check_setup, setup, init_depot, verif_clone, new_cadrage
|-- tests/      test_environment (23), test_cadrage (12),
|               test_git_history (3)
|-- exemples/ notebooks/            facultatif, non noté
|-- data/ models/ app/ ui/          avec leurs .gitkeep
```

42 fichiers suivis. Trois portent votre travail, un est généré, un est modifié.

Ce qui ne doit surtout pas s'y trouver

Ne doit pas apparaître	Pourquoi
`.venv/`	Des centaines de Mo, reconstructible en une commande
`__pycache__/`, `*.pyc`	Bytecode régénéré à chaque exécution
`.env`	Contient des secrets
`models/*` sauf `.gitkeep`	Artefacts lourds, reproductibles
Archives, PDF, captures d'écran	Rien à faire dans un dépôt de code

```
git ls-files | findstr /R "venv pycache \.env$ \.zip$"
```

La sortie doit être vide, à l'exception de `.env.example`, qui est légitime.

La liste à cocher, et la notation

Avant de quitter la salle

- ☐ Les trois fiches remplies et poussées
- ☐ Tableau du binôme complété
- ☐ `requirements.lock` présent
- ☐ Aucun fichier parasite
- ☐ `git shortlog -sn` liste les deux membres
- ☐ 0 erreur, 0 avertissement au diagnostic
- ☐ `pytest -q` affiche 38 passed
- ☐ Onglet Actions vert
- ☐ Test du clone frais passé
- ☐ Enseignant ajouté en lecture

Notation sur 10

Critère	Pts
Fiches justifiées par deux critères, rubriques remplies	4
Dépôt clonable, installation sans erreur	2
Structure et `.gitignore` conformes, `.venv/` absent de l'historique	2
Un commit par membre, messages explicites	2

Et ensuite

La semaine 2 part de ce dépôt

Vous y ajouterez `src/` et le modèle entraîné ; `models/` cessera d'être vide ; et les 38 tests d'aujourd'hui continueront de tourner à chaque push, aux côtés des quatre nouveaux.

Rien de ce que vous avez fait aujourd'hui ne sera jeté.

Documentation du kit : `MARCHE-A-SUIVRE.md`, `TESTS.md`, `AIDE-GIT.md`, `DEPANNAGE.md`
Kit : `https://github.com/larhlimi2023/sentiment-app.git`